GCI

CIP-Regel

1) Ordnungszahl: höchste Ordnungszahl = Priorität 1
2) Bei mehreren gleichen enden der Substituenten durch Adjazenz Liste. Liste vergleichen.
3) Wenn Liste gleich Nächstes
4) Länge Liste höhere Priorität

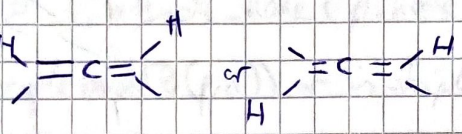
Z > E
RR/SS > RS/SR
R > S

-> Ordne Substituenten nach CIP-Regel

I) Isomerie an Doppelbindung: Z: Substituenten mit größerer Ordnung an gleicher Seite
E: sonst

II) Isomerie an tetraedrischen Kohlenstoff: R: betrachte Atom mit dem Substituenten niedrigster Ordnung von Auge wegzeigend, dann 1-2-3 im Uhrzeigersinn
S: sonst

III) Allene:



Algorithmus

- Ausgehend von chiralen Zentrum BFS der Substituenten.
- Mehrfachbindung werden durch Duplikate der Atome aufgelöst, nur einer wird in der Graphdatenstruktur weiter verfolgt, welches zudem ein Duplikat-Atom enthält. Duplikat-Atome unterscheiden sich von den ursprünglichen Atom durch gebundenen H.
- Ringe an Ringbindung geöffnet, das erste Folgeatom wird wiederholt.
- Ermittlung des vollständigen Substituentenbaumes:

DFS. Rekursiv abbruch wenn ein Knoten zum zweiten mal besucht wird!

BRUNNEN

GdCI

CIP: Alg

Datenstruktur für Substituentenbaum:

-> T.attach_node(u,v): fügt v als Kind von u hinzu

-> n.atom(): Atom zu Knoten n

-> v.node(): ersten in einem Branch erreichten Knoten von v

-> T.is_predecessor(u,x): True if x vorgänger von u

exp ∀

Tuple

```
CREATE-SUBSTITUENT-TREE(molecule graph G, atom v)
    T.attach_node(NIL, v)   //Root
    for(∀ u ∈ v.adj)
        REC-CREATE-BRANCH(u, v, G, T)   //Branch

REC-CREATE-BRANCH(atom u, atom p, Graph G, Tree T)
    node_u = T.attach_node(p.node(), u)
    if( is_doublebond((p,u)) or is_triplebond((p,u)))
        T.attach_node(p.node(), u)
        T.attach_node(node_u, p)
    if (is_triple_bond((p,u)):
        T.attache_node(p.node(), u)
    for(∀ w ∈ u.adj)
        if (w == p) continue
        if(T.is_predecessor(node_u, w))   //Ringschluss
            T.attach_node(node_u, w)
        else:
            REC-CREATE-BRANCH(w, u, G, T)
```

BRUNNEN

GrC I

CIP-Alg

- Vergleich Substituentenbäume:
  → von innen nach außen bis zwei unterschieden sind

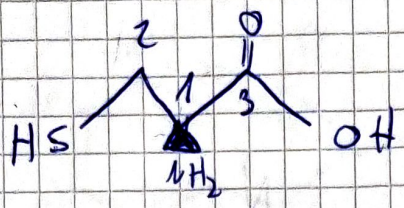
- Strategie:
  - COMPARE_BONDS: entscheidet welche Bindung zu einem Atom höherer Priorität führt
  - SORT-ADJ-LIST: sortiert die Adjazenzliste eines Atoms nach Verwendung von COMPARE-BONDS()
    ↳ beliebiges Sortierverfahren

→ Effizienzoptimierung:
  → Ergebnisse von Vergleichen sollten gespeichert werden
    ↳ merken, welche Adj.liste sortiert sind
    ↳ merken, gleiche/ungleiche Einträge
  → Baum mit Ebenen aufbauen
  → Atom-Automorphismen mit z.B. Morgan pre computed

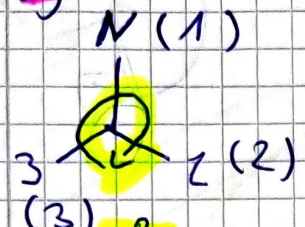
GCI

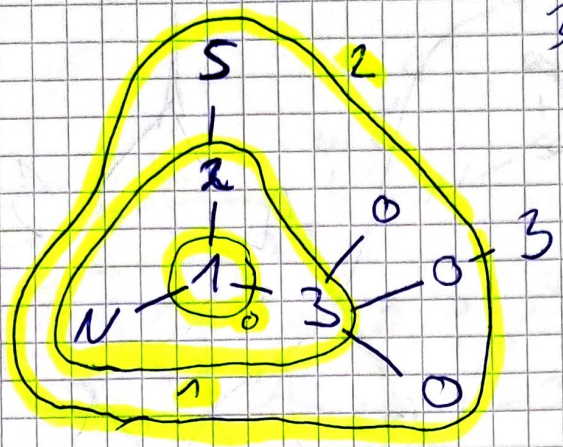
7-1 CIP

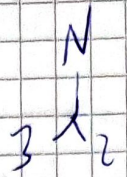
1)



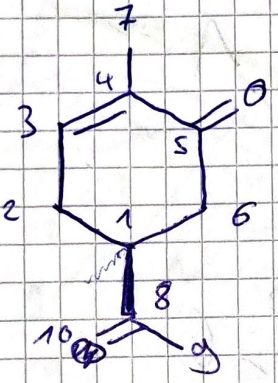
2)

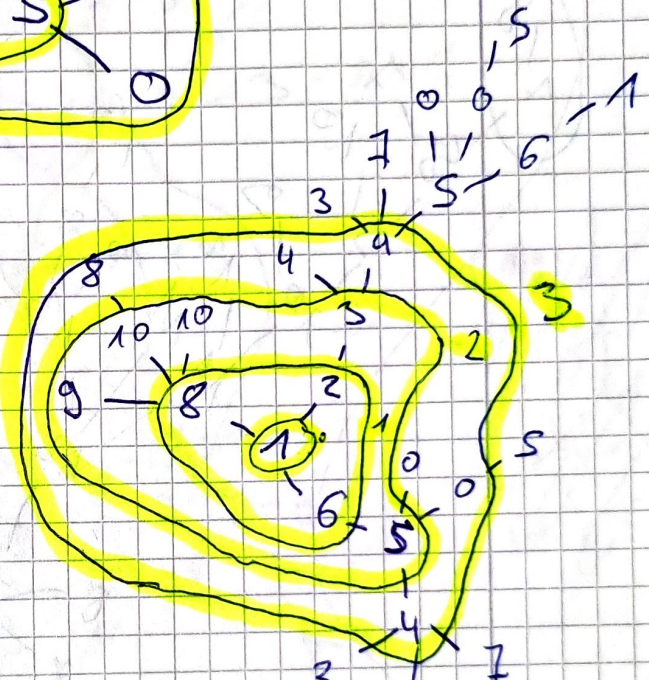


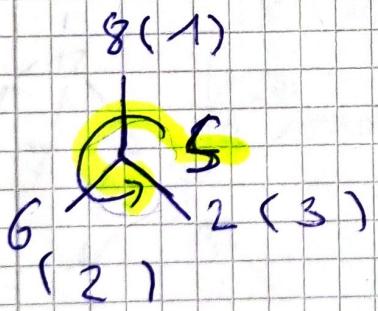


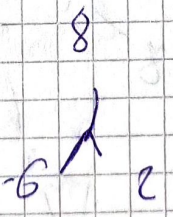


3)





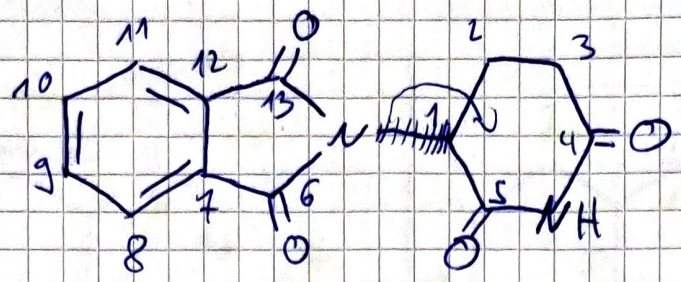


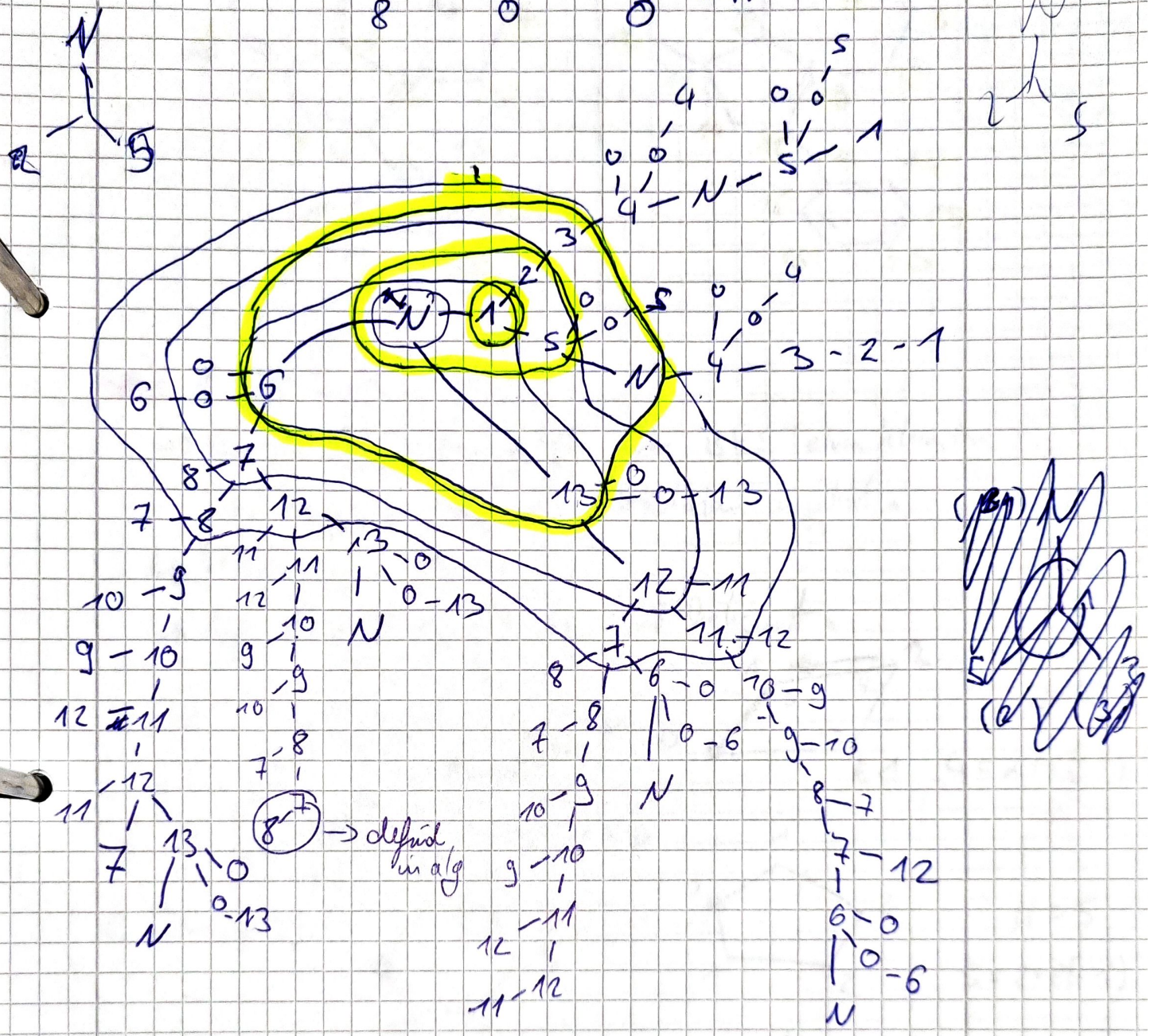


GC1

7-1 CIP

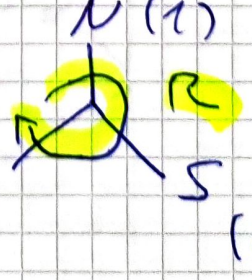
1) c)

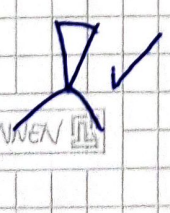


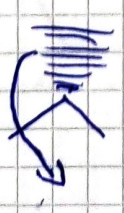


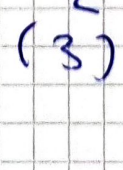
→ defiñd in alg

N(1) S(2) 2(3)







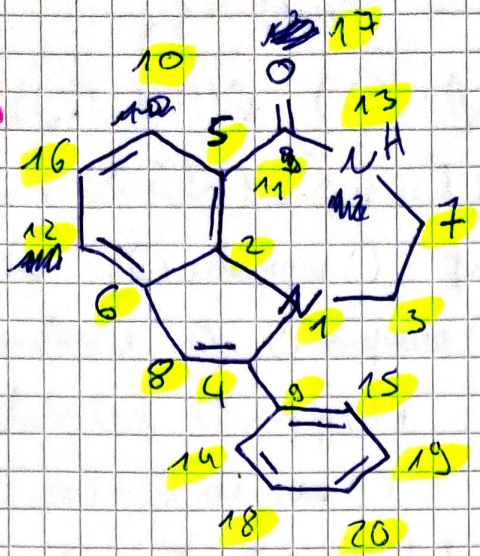


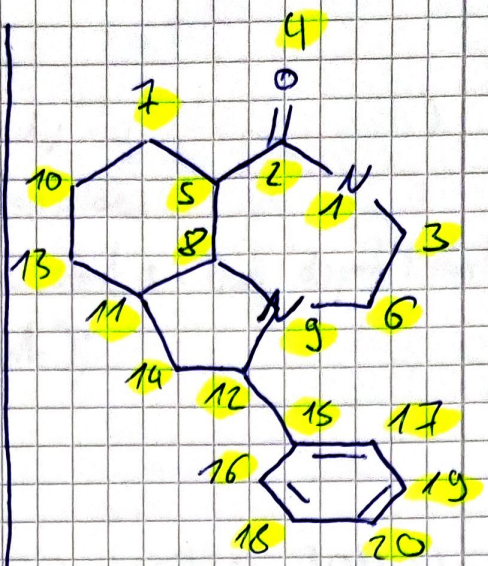
BRUNNEN

GCI

(7-2) Brüdersuche

a)



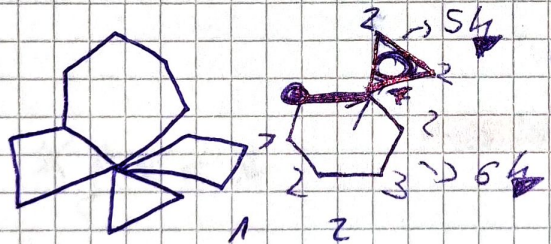


2) Pseudo Code

→ Das für ein Stadatom mittels einer BFS einen kleinsten Ring bestimmt, dem das Stadatom angehört.

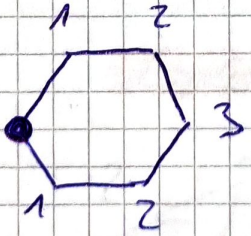
↳ Stadatom ist im Queue

↳ Ring-/Nicht-Ring-Bindung sind bekannt.

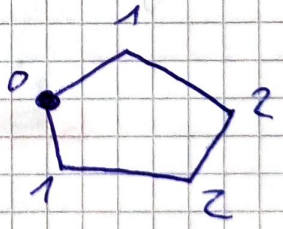


11 = 4

4 = 2+1(+1)



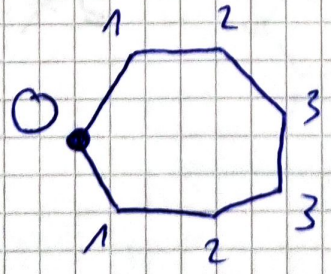
6 = 3+2(+1)

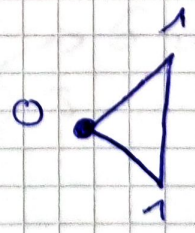


11 = 5

5 = 2+2(+1)

7 = 3+3(+1)





3 = 1+1(+1)

BRUNNEN

GC1

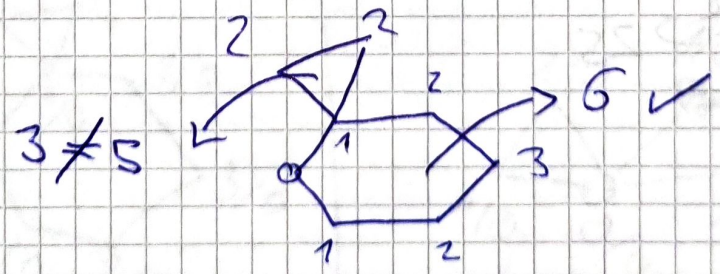
(1-2) Backtracking

2) → in Graph

```
SMALLEST_RING (G=(V,E),                    )
    for (∀v∈V): queued[v]=False
    start = Q.dequeue(); queued[start] = True; depth[start]=0
    smallest_r = 0; Q.enqueue(start);
    while(not Q.empty())
        w = Q.dequeue()
        for (∀u∈w.adj && (u,w) is RING_BOND)
            if (not queued[u])
                Q.enqueue(u); queued[u] = True; depth[u]=depth[w]+1
            else if (depth[u] ≠ depth[w]):
                length = depth[u] + depth[w] + 1
                if length < smallest_r :
                    smallest_r = length
```

→ Smallest Ring in Graph!

↳ Error



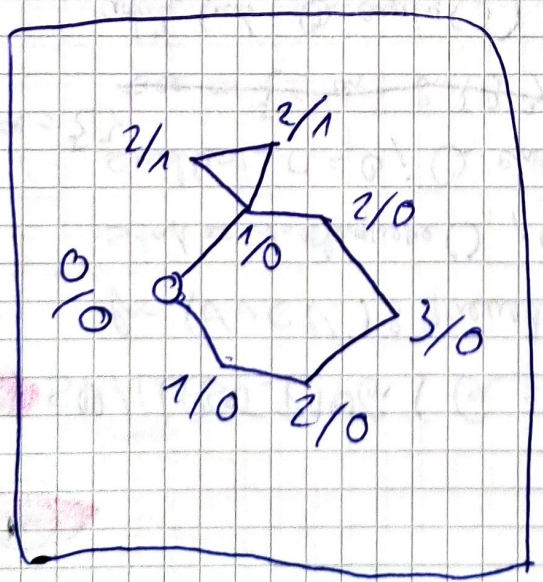
BRUNNEN

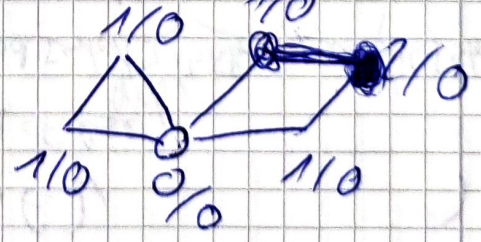
7-2)

SMALLEST_RING(G=(V,E))

2)

```
for (∀v ∈ V) : queued[v] = false
start = Q.dequeue(); queued[start] = True; depth[start] = 0
smallest_r = ∞; Q.enqueue(start)
DFS-CYCLIC-AND-ACYCLIC-BONDS(G)   // labelled Kanten mit RING or NOT
ring_edge = (ε, ε)                // & calculate low[v], ∀v ∈ V
while (not Q.empty()):
    w = Q.dequeue()
    for (∀u ∈ w.adj && (u,w) is RING_BOND):
        if (not queued[u]):
            Q.enqueue(u); queued[u] = True; depth[u] = depth[w] + 1
        else if (depth[u] ≥ depth[w]):   // not parent
            if (low[u] = low[start] = low[w]):   // gleicher Ring wie start
                length = depth[u] + depth[w] + 1
                if length < smallest_r:
                    smallest_r = length
                    ring_edge = (u,w)
```





GC1

(1-2)

3)

explore(

*: list_ring.append(w, u, depth[w], [ ]))

list_ring.apped(reverse(explore(u, w, depth[u], [ ])))

~~for (~~ remove_doubles(list_ring)

print(list_ring[i], depth[list_ring[i]]

---

explore(atom v, atom p, int l, list list):

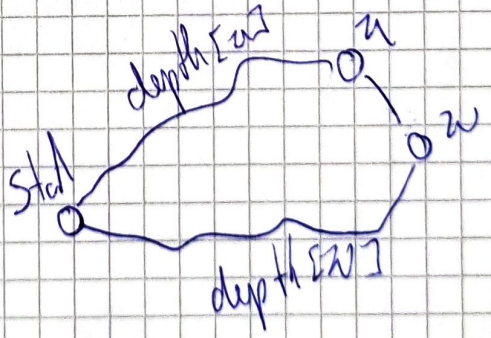
list.append(v)

l - -

if(l == 0): if(v == start): return list

else: for(x ∈ v.adj):

explore(x, v, l, list)



BRUNNEN

GCI

Zusammenhang

Gegeben: Molekülgraph

Gesucht: Zerlegung der Atome in kovalent miteinander verbundenen Atomgruppen

Zusammenhang im ungerichteten Graphen:

→ Zwei Knoten sind verbunden, ein Path von u nach v existiert

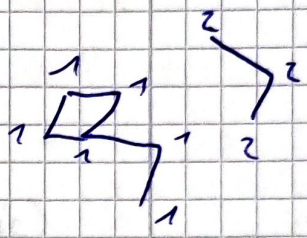
→ Verbundenheit ist Äquivalenzrelation (symmetrisch, reflexiv, transitiv)

→ Äquivalenzklassen → (CCs) Zusammenhangskomponenten

Alg (DFS)

CC[v] = # der ZHK (CC)

nof_cc = # der CCs im Graph



```
NOF-COMPONENTS(G=(V,E))
    for (∀ v∈V)  CC[v] = 0;
    nof_cc = 0;
    for (∀ v ∈V)
        if (CC[v]=0) nof_cc++; DFS_VISIT(v, nof_cc);
    return (nof_cc)

DFS_VISIT(vertex v, int cc_no)
    CC[v] = cc_no
    for (∀ u ∈ v.adj)
        if (CC[u] == 0) DFS-VISIT(u, cc_no)
```

BRUNNEN

GC1

Klassifikation einer Bindung azyklisch/zyklisch

Strategie: gegeben Bindung $e = (a, b)$. Starte einen DFS-Durchlauf an Atom a ohne Berücksichtigung von e. Teste ob b erreicht wurde.

Alg (DFS):

cc: 0 = noch nicht besucht; 1 = besucht; 2 = Zielknoten

```
IS-ACYCLIC-BOND(G=(V,E), e=(a,b))
    for (∀ v∈V) cc[v]=0
    cc[b]=2
    DFS-VISIT(a, true)
    if(cc[b]==1) return (cyclic) else return (acyclic)
DFS-VISIT(vertex v, int root)
    cc[v]=1
    for (∀ u∈v.adj)
        if(cc[u]==0 or (cc[u]==2 and not root)) DFS-VISIT(u, false)
```

DFS-Durchlauf: $O(|V|+|E|)$

Aller Bindung: $O(|E|(|V|+|E|))$ ↯

BRUNNEN

GCI

Zerlegung in Ring- und Kettenbindungen

Gegeben: Molekülgraph

Gesucht: Zerlegung der Bindungsmenge in Ringbindung (cyclic) und Kettenbindung (acyclic)

Bizusammenhang in ungerichteten Graphen:

-> Node v heißt Gelenkknoten, wenn die Entnahme von v den Zusammenhang zerstört

-> Zwei Kanten sind bizusammenhängend, wenn es einen einfachen (kein Node doppelt) Zyklus gibt, der die beiden enthält.

-> Bizusammenhang ist Äquivalenzrelation auf der Menge der Kanten; Äquivalenzklassen -> (BCCs) Bizusammenhangskomponenten -> DFS

-> BCCs sind entweder Bindung -> acyclisch

Lemma (Berechnung von Gelenkknoten mittels DFS)

I) Die Wurzel des DFS-Baums G_π ist ein Gelenkknoten, genau dann wenn der Knoten mindestens zwei Baumkanten hat.

II) Ein Blatt in G_π ist kein Gelenkknoten.

III) Ein innerer Knoten v von G_π ist ein Gelenkknoten, iff Kind s von v existiert, so dass keine Rückkante von s oder einem Nachfolger von s zu einem Vorgänger von v hat.

BRUNNEN

GdA

Vorlesung in Prof. Dr. Kothenburg

- **Zeitstempel & Lowpoints:**
  - -> Atome werden im DFS-Durchlauf in der Reihenfolge des erstmaligen Besuchs numeriert (Discovery-Zeit d)
  - -> Lowpoints: (parent ↑)

$$low[v] = \min\{d[w] \mid w = v \vee (v,w) \in E \setminus \{(v, v.\pi)\} \wedge \exists \text{Nachfahre } x \text{ von } v : (x,w) \in E\}$$

  - ↳ Kleinste discovery-Zeit eines Nodes, der von v über Baum- und Rückkante erreicht werden kann.

- **Erkennung von Gelenkknoten:**
  G ein ungerichteter Graph. $v \in V$ ist ein Gelenkknoten, iff eins von 2:
  - I) v ist Wurzel eines dfs-Baums mit mindestens zwei Kindern
  - or
  - II) v besitzt ein Kind c mit: $low[c] \geq d[v]$

- **Erkennung von azyklischen Bindungen:**
  Eine Kante $e = (v,w)$ ist azyklisch iff:
  - I) e eine Baumkante // keine Rückkante
  - and
  - II) $low[w] > d[v]$

- **Berechnung Lowpoints mit DFS:**
  - initialisiere $low[v]$ mit $d[v]$
  - minimiere über die Discovery-Zeit bei Rückkanten.
  - propagiere das Ergebnis jeweils zurück an die aufrufende Funktion und minimiere

BRUNNEN

GCI

Verlegung in Ring-/Kettelbindung

d: Discovery Zeit im DFS ; low: Lowpoint-Nummer

cyclic: True, if Ringbindung else False

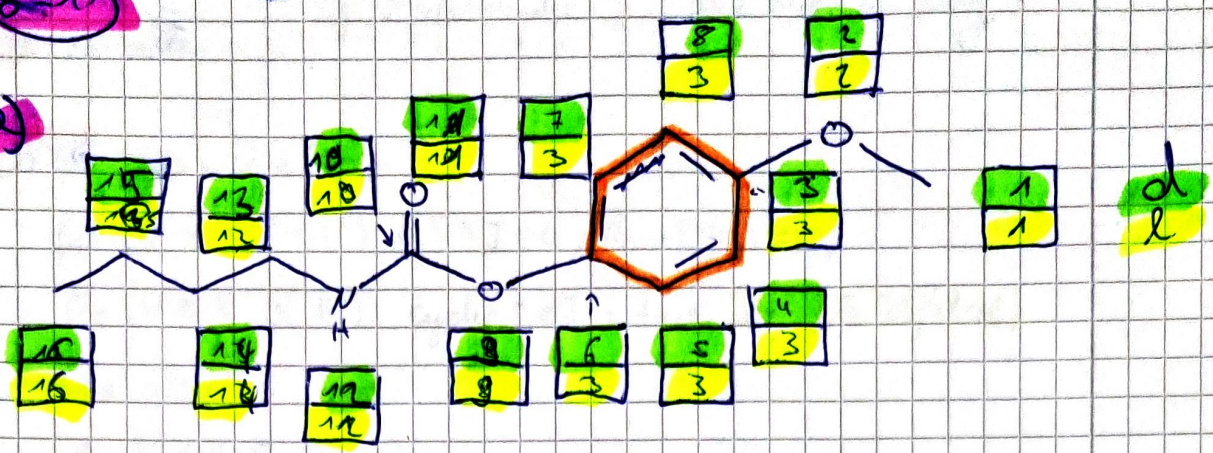
```
DFS-CYCLIC-AND-ACYCLIC-BOUNDS(G):
    for (∀ u ∈ G.V) d[u]=0; low[u]=0;
    for (∀ e ∈ G.E) cyclic[e]=true;
    time=0
    for (∀ u ∈ G.V)
        if (d[u]==0) DFS-VISIT(u, nil);

DFS-VISIT(u, parent)          //Keine zyklischer Bindung
    time++; d[u]=time
    low[u]=d[u]
    for (∀ v ∈ u.adj)         //Sondern der Adjazenzliste
        if (d[v]==0)          //Baumkante
            DFS-VISIT(v, u)
            low[u]=min(low[u], low[v])   //Min über Nachfolger
            if (low[v]>d[u]) cyclic[(u,v)]=false;
        else if (v ≠ parent)  //Rückkante
            low[u]=min(low[u], d[v])     //Min über Rückkanten.
```

BRUNNEN

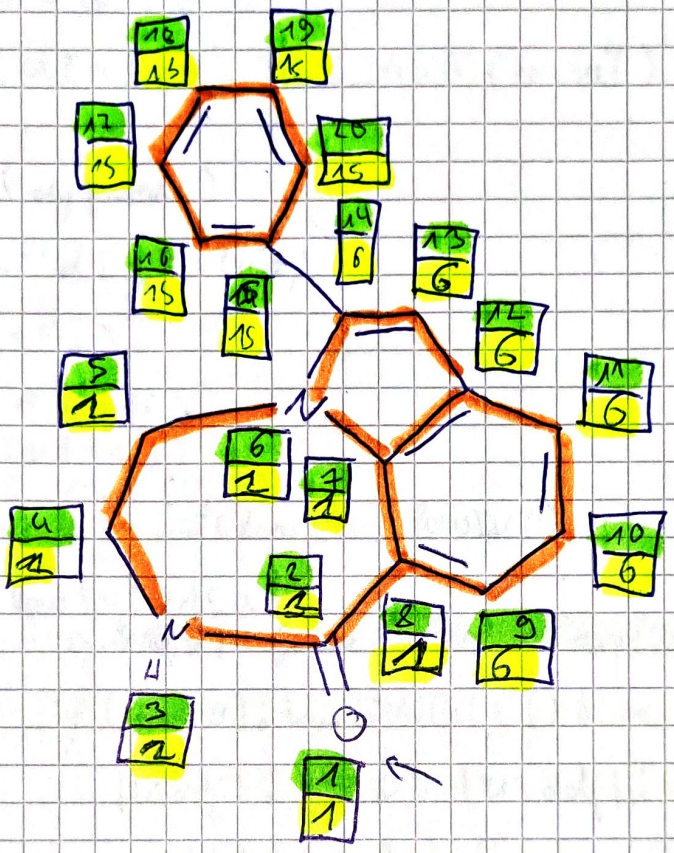
8.1)

a)



d
l

b)



d
l

BRUNNEN

GdI

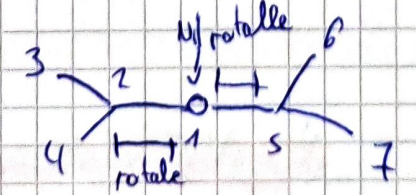
8-1)

I) Nicht terminal
II) Einfach zsh
III) Nicht im Ring!

2) Rotalle

```
DFS_BONDS(G)
    for(∀x ∈ G.V) d[x] = 0; low[x] = 0;
    for(∀e ∈ G.E) cyclic[e] = true; rotalle[e] = false;
    time = 0;
    for(∀u ∈ G.V)
        if(d[u] == 0) DFS_VISIT(u, nil);

DFS_VISIT(u, parent)
    time++; d[u] = time;
    low[u] = d[u]
    for(∀v ∈ u.adj)
        if(d[v] == 0)
            DFS_VISIT(v, u)
            low[u] = min(low[u], low[v])
            if(low[v] > d[u])
                cyclic[v,u] = false; // No ring (v,u)
                if((v,u).is_SINGLE() &&
                   |v.adj| != 1 && |u.adj| != 1)
                    rotable[(v,u)] = true
        else if(v ≠ parent)
            low[u] = min(low[u], d[v])
```



BRUNNEN

GCI

Ullmann Algorithmus

Frage:
Gegeben: Molekülgraph G und ein Fragment F
Gesucht: • Kommt F als Teilstruktur in G vor?
• Wie häufig kommt F in G vor?
• Abbildung von F in G, welches Atom wird durch welches Atom repräsentiert.

Varianten: • Dürfen Ketten auf Ringbildung abgebildet werden?
• zugeordneter Teilgraph von G ein induzierter Subgraph?

Subgraph-Isomorphie:
- Sei $G=(V,E)$ ein graph, dann ist $G'=(V',E')$ mit $V' \subseteq V$, $E' \subseteq E \cap (V' \times V')$ ein Subgraph von G.
Ist $E' = \{(u,v) \in E \mid u,v \in V'\}$, so ist G' ein induzierter Subgraph von G

- Graph-Isomorphie: G & F sind isomorph, iff eine bijektive Abbildung $m: V_F \to V_G$ existiert:

$$(u,v) \in E_F \Leftrightarrow (m(u), m(v)) \in E_G$$

- Subgraph-Isomorphie-Problem: G & F, existiert eine bijektive Abbildung $m: V_F \to V_{G'}$, $V_{G'} \subseteq V_G$:
→ $(u,v) \in E_F \Leftrightarrow (m(u), m(v)) \in E_{G'}$ (induzierter Subgraph)
→ $(u,v) \in E_F \Rightarrow (m(u), m(v)) \in E_{G'}$ (Subgraph)

→ NP-vollständiges Problem

BRUNNEN

GrCi

Ullmann - Algorithmus

In: Subgraph $F = (V_F, E_F)$, f nodes, Adjazenzmatrix A_F
Graph $G = (V_G, E_G)$, g nodes, Adjazenzmatrix A_G

Zentrale Datenstrukturen:

→ Verwaltung der bijektiven Abbildung m als Matrix

→ Zuordnungsmatrix $P = (p_{ij})\ 1 \le i \le f,\ 1 \le j \le g$

$$p_{ij} = \begin{cases} 1 : \text{Knoten } i \text{ aus } F \to \text{Knoten } j \text{ aus } G \\ 0 : \text{else} \end{cases}$$

→ Condition (Permutationsmatrix):

I) $\forall i : \sum_j p_{ij} = 1$

II) $\forall j : \sum_i p_{ij} \le 1$

Überprüfen Subgraph-Isomorphie (induziert):

- $(u,v) \in E_F \Rightarrow (\Leftrightarrow)\ (m(u), m(v)) \in E_G$
- $P A_G P^T \geqslant (=)\ A_F$

Die Initial-Matrix M:

→ Zeilen: Atome des Subgraph F

→ Spalten: Atome des Graphen G

$$m_{ij} = \begin{cases} 1 : i \text{ aus } F \text{ kann } j \text{ aus } G \text{ zugeordnet werden} \\ 0 : \text{else} \end{cases}$$

→ Überprüfbare Bedingungen:

I) Element (or anderen Prop)

II) Hybridisierung

III) Knotengrad: $\deg(v_i; F) \le \deg(v_j; G)$

BRUNNEN

GCI

Ullman-Alg

Basis-Strategie:

- Suche Permutationsmatrix P mit $PAG P^T = A_F$ und $p_{ij} \leq m_{ij}$
- Backtracking: wandle zeilenweise Initial-Matrix → Permutationsmatrix um
- k-ter Rekursionsabschnitt (Knoten auf Ebene k im Suchbaum):
  - → $\forall 1 \leq i < k: \sum_j p_{ij} = 1 \;\wedge \sum_{i=1,\dots,k-1} p_{ij} \leq 1$
  - → Speichere belegte Spalten in Vektor u

$$u_j = \sum_{i=1,\dots,G-1} p_{ij}$$

  - → Iteriere durch alle Spalten $1 \leq j \leq g$:

    falls $m_{kj} = 1$ und $u_j = 0$:

    setze $p_{kj} = 1$ ⇒ restliche Zeile k auf 0

    Backtracking gehe zu $k+1$

Laufzeit (Basis-Strategie):

- → Anzahl von $\#m_{ij} = 1$ ab $\;; e_i = \sum_j m_{ij}$
- → $T_u\text{-Base} \sim \prod_{i=1}^{g} e_i = \prod_{i=1}^{g} \sum_{j=1}^{g} m_{ij}$

→ e.g.: 2 Alternativen pro Zeile: $T_u\text{-Base} \sim 2^g$

Optimierungsoptionen:

I) Prüfe für die ersten $k-1$ Zeilen, ob sie die Subgraph-Kanonie erfüllen. Falls nicht stopp

II) Prüfe für die Zeilen $k, \dots, f$, jede 1 ob diese noch zur Erweiterung des Matchings beitragen kann. Falls nicht lösche die 1 (Ullman-Strategie)

BRUNNEN

GrCl

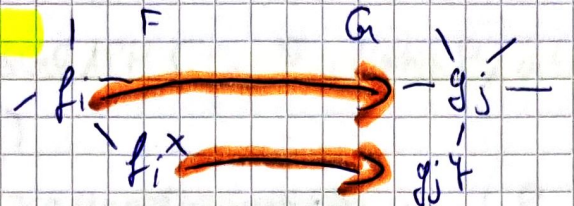
Ullmann-Alg

Notwendige Bedingung für $m_{ij} = 1$ in einer Subgraph-Permutation (Ullmann-Kriterium)

→ Seien $f_i^0, \dots, f_i^x, \dots$ adjacent zu f_i

Seien $g_j^0, \dots, g_j^y, \dots$ adjacent zu g_j

→ dann muss für jedes f_i^x in F ein Atom g_j^y existieren mit $m_{xy} = 1$



Satz:
→ Erfüllt M das Ullmann-Kriterium, so ist M ein gültiges Matching → erfüllt Subgraph-Isomorphie-Kriterium (weiß induziert)

Alg

```
ULLMANN-REFINE (Gr, F, Matrix P, int k)
  do
    change = false          // true, if M changed
    for (∀ pij with j > k and pij = 1)
      valid = true          // true, if pij fulfills condition
      for (∀ fi^x ∈ fi.adj)
        found = false       // true, if fi^x can be mapped
        for (∀ gj^y ∈ gj.adj)   // true, if fi^x can be
          if (pxy == 1) found = true; break;
        if (not found) valid = false; break;
      if (not valid)
        pij = 0; change = true;
        if (pik == 0 for each 1 ≤ k ≤ |G.V|) return (false)
  until (not change)        // till <=> while (change)
  return (true).
```

BRUNNEN

Ullmann-Algorithmus

```
ULLMANN-ALG(G1, F)
    M = CALCULATE-INITIAL-MATRIX(G1, F)
    for (j=1 to |G1.V|) U[j]=0
    ENUMERATE-SUBGRAPH(G1, F, M, 0, U)

ENUMERATE-SUBGRAPH(G1, F, Matrix P, int h, Vector U)
    if (h==|F.V|)
        ~~if (CHECK-ISOMORPHIC(G1,F))~~ output P   // subgraph found
    else:                                          // modify rows h+1
        for (j=1 to |G1.V|)
            if(P_{h+1,j}==1 and u_j==0)            // store in compatible
                P_save=P                           // and not used yet
                for (l=1 to |G1.V|) P_{h+1,j}=0
                P_{h+1,j}=1; u_j=1;
                if (ULLMANN-REFINE(G1,F,P,h+1))    // execute refinement
                    ENUMERATE-SUBGRAPH(G1,F,P,h+1,U)
                P=P_save; u_j=0;
```

Weitere Optimierung:

→ Verwende Zahlen mit niedrigster 1-Dichte in der h-ten Zeile

→ sobald M_{hj}=1, lösche alle 1-Einträge m_{ij} mit l>h, U wird nicht mehr benötigt

BRUNNEN

GCI

8-1 Subgraphisomorphie

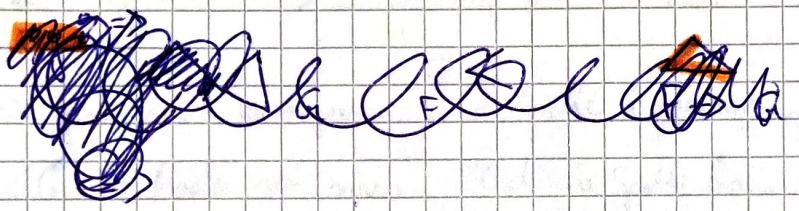
1) $f: V_F \to V_G' \subseteq V_G$ (1)

-> bijektive Abbildung bildet jeden Knoten aus F genau auf einen Knoten in G ab ~~, wobei G im allgemeinen~~ . Die Abbildungen V_G' bilden eine Teilmenge von G

$\forall v,w \in V_F : e(v,w) \in E_F \Rightarrow (\Leftarrow)\ e(f(v),f(w)) \in E_G$ (2)

-> (=>) wenn v & w eine Kante (v,w) in F bilden dann bilden die Abbildungen f(v) & f(w) eine Kante (f(v),f(w)) in G.

- (<=>) -11- & jede Kante zwischen zwei Abbildungsenden Knoten $(f(v),f(y))$ f(v) & f(w) in G & impliziert eine Kante (v,w) in F



eg: F

G: bei => erfüllt bei <= nicht erfüllt!

BRUNNEN

8-2)

2) $M' = [m_{ij}] \mid 1 \leq i \leq a \wedge 1 \leq j \leq b$ (3)

→ Die Matrix braucht so viele Zeilen wie das Norden in F gültig sind so viele Spalten wie es Norden in G gibt.
Damit jeder Knoten aus F einen / mehr Knoten in G zugeordnet werden kann.

$m_{ij} = (1 \text{ für } deg(v_j, G) \geq deg(v_i, F) \wedge (elem(v_j, G) = elem(v_i, F)), \text{ else } 0$ (4)

→ 0 Gleiche Elemente, wenn Knoten in G mindes so viele Kanten hat wie Knoten in F wegen (2)

$\sum_i m_{ij} = 1 \quad \wedge \quad \sum_j m_{ij} \leq 1$ (5)

→ Jede Zeile muss genau einer Spalte zugeordnet sein da F im G zu finden sein muss. Und eine Spalte kann nur einer Zeile zugeordnet sein und $|V_G| \geq |V_F|$

(2) & (4): $\Rightarrow$; ~~$deg(v_j, G) \geq deg(v_i, F)$~~
$\Leftrightarrow$; ~~$deg(v_j, G) = deg(v_i, F)$~~

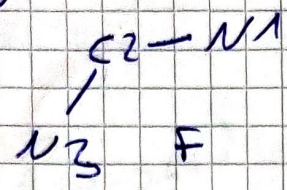
BRUNNEN

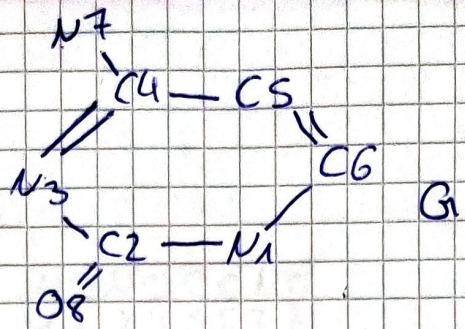
GC1

Nach Ref Vor Rek

(8-2)

3)





G

INIT:

0		N	C	N	C	C	C	N	O
i		1	2	3	4	5	6	7	8
N	1	1	0	1	0	0	0	1	0
C	2	0	1	0	1	1	1	0	0
N	3	1	0	1	0	0	0	1	0
F		0	0	0	0	0	0	0	0

h=0 → Berechne h+1

1		N	C	N	C	C	C	N	O
		1	2	3	4	5	6	7	8
N	1	1	0	0	0	0	0	0	0
C	2	0	1	0	0	0	1	0	0
N	3	1	0	1	0	0	0	0	0
F		1	0	0	0	0	0	0	0

$f_i = C6$

$f_i^x = (N1) N3$

$g_i = C2$

$g_i^x = (N7), N3$

h=1 → h+1

4		N	C	N	C	C	C	N	O
		1	2	3	4	5	6	7	8
N	1	1	0	0	0	0	0	0	0
C	2	0	0	0	0	0	1	0	0
N	3	1	0	0	0	0	0	0	0
F		1	0	0	0	0	1	0	0

h=1 → h+1

2		N	C	N	C	C	C	N	O
		1	2	3	4	5	6	7	8
N	1	1	0	0	0	0	0	0	0
C	2	0	1	0	0	0	0	0	0
N	3	1	0	1	0	0	0	0	0
F		1	1	0	0	0	0	C	C

$f_i = N3$

$f_i^x = C2$

$g_i = N1$

$g_i^y = C2, C6$

h=2 → h+1

5 No Refine and Rekursion call

3		N	C	N	C	C	C	N	O
		1	2	3	4	5	6	7	8
N	1	1	0	0	0	0	0	0	0
C	2	0	1	0	0	0	0	0	0
N	3	0	0	1	0	0	0	0	0
F		1	1	1	0	0	0	0	0

h=0 → h+1

6'		N	C	N	C	C	C	N	O
		1	2	3	4	5	6	7	8
N	1	0	0	1	0	0	0	0	0
C	2	0	1	0	1	0	0	0	0
N	3	1	0	1	0	0	0	1	0
F		0	0	1	0	0	0	0	0

h=1 → h+1

7		N	C	N	C	C	C	N	O
		1	2	3	4	5	6	7	8
N	1	0	0	1	0	0	0	0	0
C	2	0	1	0	0	0	0	0	0
N	3	1	0	1	0	0	0	0	0
F		0	1	0	1	0	0	0	0

		N	C	N	C	C	C	N	O
		1	2	3	4	5	6	7	8
		Stopp							

8		N	C	N	C	C	C	N	O
		1	2	3	4	5	6	7	8
N	1	0	0	1	0	0	0	0	0
C	2	0	1	0	0	0	0	0	0
N	3	1	0	0	0	0	0	0	0
F		1	1	1	0	0	0	0	0

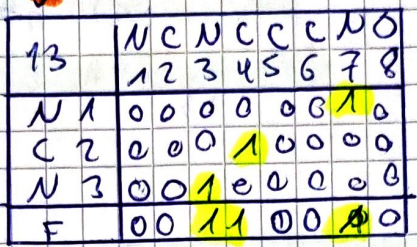
9		N	C	N	C	C	C	N	O
		1	2	3	4	5	6	7	8
N	1	0	0	1	0	0	0	0	0
C	2	0	0	0	1	0	0	0	0
N	3	0	0	0	0	0	0	1	0
F		0	0	1	1	0	0	0	0

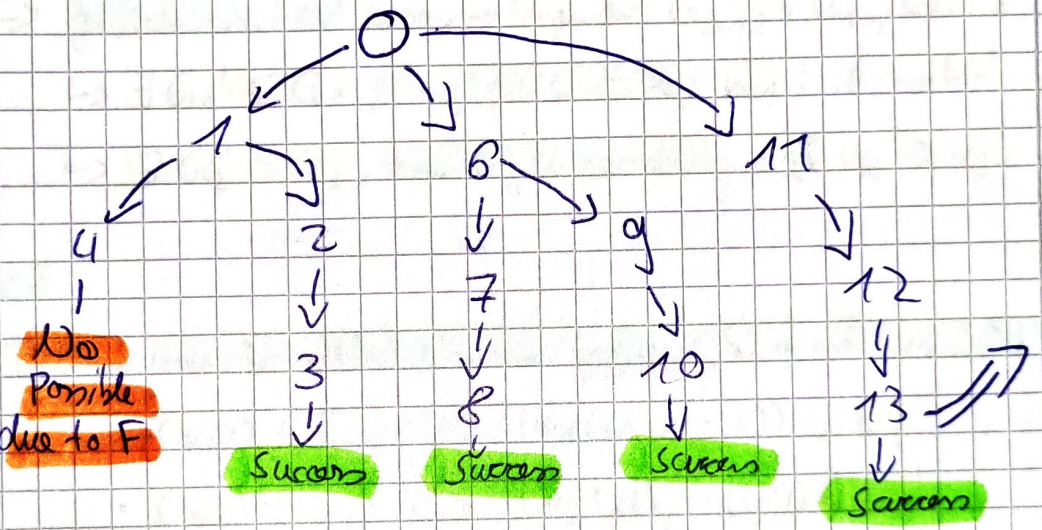
10		N	C	N	C	C	C	N	O
		1	2	3	4	5	6	7	8
N	1	0	0	1	0	0	0	0	0
C	2	0	0	0	1	0	0	0	0
N	3	0	0	0	0	0	0	1	0
F		0	0	1	1	0	0	1	0

GdCI

8.2

3)





4)

#Rekursionen ohne Refinement

$$T = \prod_{i=1}^{3} e_i = \prod_{i=1}^{3} \sum_{j=1}^{8} m_{ij} = 3 \cdot 4 \cdot 3 = 12 \cdot 3 = 36$$

ohne Refinement

36 >> 13 :o

BRUNNEN

GCI

Maximale gemeinsame Teilstruktur

Frage: Gegeben zwei Moleküle A & B, welches ist der größte gemeinsame Teil (Subgraph) von A und B?

Varianten:
- Subgraph oder induzierter Subgraph?
- zusammenhängender oder nicht-zusammenhängender Subgraph?
- Zuordnung Ring auf Kette erlaubt?
- Größe (#Atome, #Bindg, ...)?

MCS-Problem:

(Node/Edge)

-> gegeben Graphen $G_1 = (V_1, E_1)$ und $G_2 = (V_2, E_2)$ mit Labels.

-> gesucht wird ein zusammenhängender $G_{12} = (V_{12}, E_{12})$:

↳ $\exists\, G_1' \subseteq G_1$, eine bijektive Abbildung $m_1: V_{12} \to V_1'$

↳ $\exists\, G_2' \subseteq G_2$, eine bijektive Abbildung $m_2: V_{12} \to V_2'$

sind:

I) Maximal Connected Common Induced Subgraph (MCCIS)
- $(u,v) \in E_{12} \Leftrightarrow (m_1(u), m_1(v)) \in E_1'$
- $(u,v) \in E_{12} \Leftrightarrow (m_2(u), m_2(v)) \in E_2'$
- $|V_{12}|$ maximal

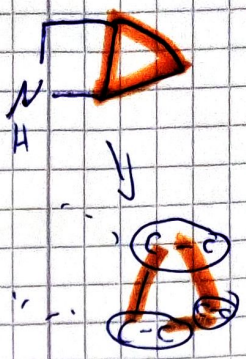
II) Maximal Connected Common Edge Subgraph (MCCES)
- $(u,v) \in E_{12} \Rightarrow (m_1(u), m_1(v)) \in E_1'$
- $(u,v) \in E_{12} \Rightarrow (m_2(u), m_2(v)) \in E_2'$
- $|E_{12}|$ maximal

-> MCS-Problem ist NP-schwer

BRUNNEN

GrCI

Maximal gemeinsame Teilstruktur



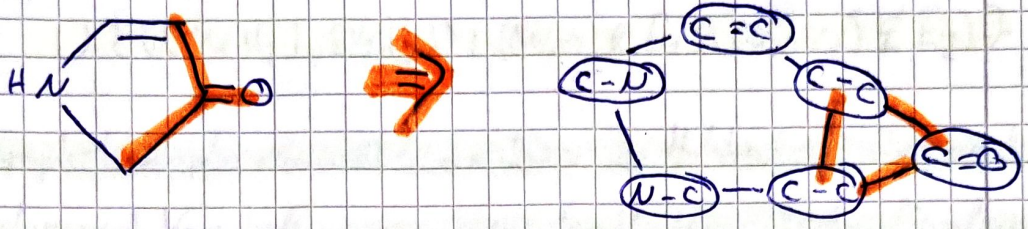
MCES-Problem $\overrightarrow{\in O}$ MCIS-Problem

Def (Linie Graphs):

Sei $G_1 = (V, E)$ ein Graph, dann $G_L = (V_L, E_L)$:

$V_L = E$

$E_L = \{(e_1, e_2) \in E \times E \mid \exists v \in V : v \in e_1 \cap e_2\}$

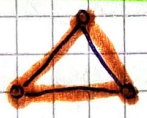


Def (ΔY-Austausch):

K_3: der vollständige Graph bestehend aus 3 Knoten

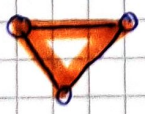
$K_{1,3}$: der vollständige bipartite Graph mit Knoten $U \cup V$, $|U| = 1$, $|V| = 3$

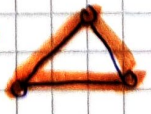
Sei G_1 ein Graph, der Tausch eines K_3-Subgraphen durch ein $K_{1,3}$-Subgraphen oder umgekehrt wird als ΔY-Austausch bezeichnet.

K_3:  $\Leftarrow$ ΔY-Austausch $\Rightarrow$ $K_{1,3}$:

Whitney:

Tritt zwischen zwei Graphen G_1 und G_2 kein ΔY-Austausch auf, so entspricht eine Isomorphie auf zwei Linien-Graphen G_{L1} und G_{L2} einer Kanten-Isomorphie auf den ursprünglichen Graphen G_1 und G_2

K_{3L} 

$K_{1,3L}$ 

Praxis: ein Molekül muss einen 3 Ring enthalten;

H Bindung in Subgraph identisch; H Atome unterscheiden sich!;

selten in der Chemie

BRUNNEN

GCI

MCS in Produktgraph

((Knoten -) Produktgraph / Kompatibilitätsgraph):

2 * Graphen $G_1 = (V_1, E_1)$ und $G_2 = (V_2, E_2)$.

Produktgraph von $G_1 \& G_2$ $G_p = (V_p, E_p)$:

$V_p = \{ [u_1, u_2] \mid u_1 \in V_1 \wedge u_2 \in V_2 \wedge label(u_1) = label(u_2) \}$ //Matches

$E_p = \{ ([u_1, u_2], [v_1, v_2]) \mid ((u_1, v_1) \in E_1 \wedge (u_2, v_2) \in E_2) \}$ c-Kanten

$\cup \{ ([u_1, u_2], [v_1, v_2]) \mid ((u_1, v_1) \notin E_1 \wedge (u_2, v_2) \notin E_2) \}$ d-Kanten

-> Subgraph-Isomorphie kann über eine Menge von Matches beschrieben werden.

-> Matches sind Kompatible wenn zwischen den Knoten in beiden Graphen eine Kante existiert oder in beiden Graphen keine Kante existiert.

-> $M \subseteq V_p$ beschreibt dann Subgraph-Isomorphie, wenn alle Matches in M zueinander kompatible sind.

Isomorphe Subgraphen und Cliquen

Seien G_1, G_2 Graphen, G_p ihr Produktgraph. Menge $M \subseteq V_p$ beschreibt isomorphe Subgraphen, genau dann wenn M eine Clique in G_p ist.

Max-Clique-Problem - NP-Schwer

BRUNNEN FH

GrCl

Max-Clique-Problem

Gegeben: ungerichteter Graph $G=(V,E)$

Gesucht: maximale Clique von G, häufig auch Menge nicht erweiterbarer Cliquen

Idee:
- → baue Clique auf
- → $C \subseteq V$; prüfe ob alle Knoten paarweise verbunden sind
- → C & C'; $C \cap C' = I$, $I \neq \emptyset$:
  - ↳ I bereits eine Clique → müssen Knoten aus I nicht überprüft werden
  - ↳ I bereits keine Clique → C & C' keine Cliquen

```
CLIQUE-BACKTRACK (G, set I, int d, set M)
    if (d == |V|+1)  // Clique gefunden
        if (|I| > |M|) M = I
    else
        // I Clique wird nicht erweitert
        v_d = select d-th node from G.V  // Fall 1: Knoten v_d erweitert
        is_clique = true
        for (each u ∈ I)
            if (u ∉ v_d.Adj) is_clique = false;
        if (is_clique) CLIQUE_BACKTRACK (G, I ∪ {v_d}, d+1, M)
        // Fall 2: Clique wird nicht um v_d erweitert
        CLIQUE_BACKTRACK (G, I, d+1, M)
```

BRUNNEN

GCI

Branch & Bound-Alg für Clique

→ untere Schranke L:

→ max # Knoten/Clique in allen bisher gefundenen Cliquen

→ Teilmengen sind bereits Cliquen → können zur Verbesserung von L herangezogen werden

→ Obere Schranke an Knoten v auf Ebene d mit Teillösung I $u(d, I)$:

→ alle Cliquen unter v enthalten die Knoten aus I

→ I kann max um $n-d$ erweitert werden:

$\Rightarrow u(d, I) = |I| + n - d$

→ Verbesserung: Bestimme Knotenmenge X von Knoten, die nicht in I sind, aber zu allen Knoten aus I adjacent sind:

$\Rightarrow U(I, X) = |I| + |X|$

```
CLIQUE-BB(G, sel I, sel X, int d, sel M):
  if (d == |G.V| + 1)        // Clique gefunden
    if(|I| > |M|) M = I
  else
    v_d = select d-th node from G.V   // lokale Schranke U = |I| + |X|,
                                      // untere Schranke ist |M|
    if(|I| + |X| < |M|) return
    if v_d ∈ X) CLIQUE-BB(G, I ∪ {v_d}, X ∩ v_d.adj, d+1, M)
    CLIQUE-BB(G, I, X \ {v_d}, d+1, M)
```

BRUNNEN

GdA

Brau-Kerbosch-Alg

-> Strategie: Erzeugung der Liste aller maximalen Cliquen, jede tritt nur einmal auf

-> Datastructure:

-> Menge compsub(I): aktuelle Cliquen im Algorithmus

-> Menge candidates(X): Knoten, zum die die Clique erweitert werden kann

-> Menge not(N): Knoten, die bereits bearbeitet wurden d.h. zum die die Clique erweitert werden hätte aber nicht erweitert werden darf

```
CLIQUE-BK ( G, Set I, Sel X, Sel N)
    if ( X == ∅ )    //und extendable
      if( N == ∅) output I    // Clique is maximal
    else
      for ( ∀n ∈ N )
        if ( n.adj ⊇ X ) return;
      for ( ∀x ∈ X )                          ohne I
        CLIQUE-BK(G, I ∪ {x}, X ∩ x.adj, N ∩ x.adj);
        X = X \ {x}
        N = N ∪ {x}
```

BRUNNEN

GCI

Clique -> Berechnung des MCCIS

-> BK-Alg löst dieses MCIS-Problem.

-> Mit Line-Graphen und dem Satz von Whitney können MCES-Probleme gelöst werden

-> Der BK-Algorithmus berechnet eine Menge Knoten mit einander kompatiblen Adjazenzen, die Menge bildet aber nicht einen zshgd Subgraph.

Def (c-Zusammenhängend):

Eine Clique $C=(V,E)$ mit $E=E_C \cup E_d$ im MCS-Kontext heißt c-zusammenhängend, iff der Teilgraph (V,E_C) zusammenhängend ist.

Satz (Zshgd MCS):

Eine Clique C im Produktgraphen beschreibt einen gemeinsamen zshgd Teilgraph, iff C ist c-zusammenhängend

Erweiterung

- n-Kanten: verbinden zwei inkompatible Knoten
- Es gibt viel mehr d-Kanten als n-Kanten.
  - Auf das nicht-Aufheben von n-Kanten geprüft (ol implizit)
- C: Knoten, die über c-Kante mit Clique I verbunden sind
- Erweiterung mit $X \cap C$, damit I immer c-zshgd.
- Die Not-Menge wird ebenfalls mit C geschnitten

GC1

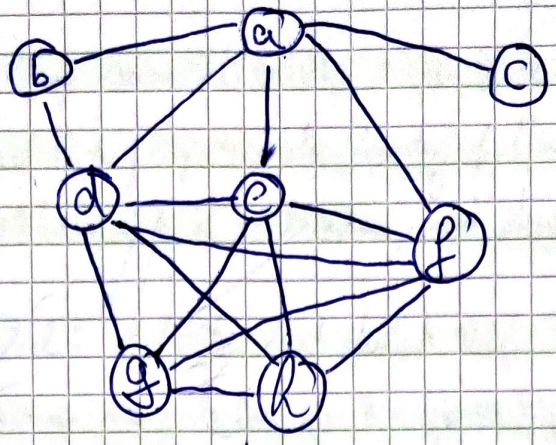
BK - F

```
Q - CLIQUE-BK (G, I, X, C, N)
    if (X ∩ C == ∅)                    //ind extendable
        if (N ∩ C == ∅) output I       //max Clique
    else
        for (∀ u ∈ N ∩ C)
            if (u.Adj ⊇ X) return;
        for (∀ x ∈ X ∩ C)
            if (I == ∅)
                C - CLIQUE-BK (G, {x}, X ∩ x.Adj, C ∪ x.Adj_C, N ∩ x.Adj)
            else
                C - CLIQUE-BK (G, I ∪ {x}, X ∩ x.Adj, C ∪ x.Adj_C, N ∩ x.Adj)
            X = X \ {x}
            N = N ∪ {x}
```

BRUNNEN

GaCl

5-2) Bron-Kerbosch



a)

$C_1 = \{a, c\}$, $|C_1| = 2$

$C_2 = \{a, b, d\}$, $|C_2| = 3$

$C_3 = \{a, d, e, f\}$, $|C_3| = 4$

$C_4 = \{d, e, f, g, h\}$, $|C_4| = 5$

↳ 4 Maximale Cliquen!

ac → ✓

a b d → ✓

Rekursionsebene

RE	R0	R1	R2	R3	R4	R5	R6	R7	R8

Start:

$I = \{a, d, e\}$, $X = \{f\}$, $N = \{\emptyset\}$

$I = \{a, d, e, f\}$; $X = \{\emptyset\}$, $N = \{\emptyset\}$ → Output

$I = \{a, d, e\}$, $X = \{\emptyset\}$, $N = \{f\}$ → Return

$I = \{a, e\}$, $X = \{f\}$, $N = \{d\}$ → Return

$I = \{a, f\}$, $X = \{\emptyset\}$, $N = \{d, e\}$ → Return

$I = \emptyset$, $X = \{b, c, d, e, f, g, h\}$, $N = \{a\}$

$I = \{b\}$, $X = \{d\}$, $N = \{a\}$ → Return!

$I = \emptyset$, $X = V \setminus \{a, b\}$, $N = \{a, b\}$

$I = \{c\}$, $X = \emptyset$, $N = \{a, b\}$ → Return!

$I = \emptyset$, $X = V \setminus \{a, b, c\}$, $N = \{a, b, c\}$ ↓ $N \cap d.adj$

$I = \{d\}$, $X = \{e, f, g, h\}$, $N = \{a, b\}$

$I = \{d, e\}$, $X = \{f, g, h\}$, $N = \{a\}$

$I = \{d, e, f\}$, $X = \{g, h\}$, $N = \{\emptyset\}$

$I = \{d, e, f, g\}$, $X = \{h\}$, $N = \{\emptyset\}$

$I = \{d, e, f, g, h\}$, $X = \{\emptyset\}$, $N = \{\emptyset\}$ → Output

BRUNNEN